1. Add Data to Existing Table

Query

```
INSERT INTO Employee VALUES
(113,'Ajay',67000,'2025-02-15',2),
(114,'Meera',72000,'2025-01-20',3),
(115,'Suresh',58000,'2025-03-05',4),
(116,'Kavita',88000,'2024-10-12',3),
(117,'Arjun',62000,'2025-04-25',2);
```

Output

```
Query OK, 5 rows affected
```

---

2. Window Function - RANK()

Query

```
SELECT emp_id,
       emp_name,
       salary,
       RANK() OVER(ORDER BY salary DESC) AS salary_rank
FROM Employee;
```

Output

Emp ID	Name	Salary	Rank
106	Neha	95000	1
116	Kavita	88000	2
104	Sneha	85000	3
109	Karan	80000	4
102	Priya	75000	5
114	Meera	72000	6
107	Rohit	70000	7
113	Ajay	67000	8
103	Rahul	65000	9
117	Arjun	62000	10
108	Pooja	60000	11
115	Suresh	58000	12
110	Anjali	55000	13
101	Amit	50000	14
105	Vikas	45000	15

---

3. Window Function - ROW_NUMBER()

Query

```
SELECT emp_id,
       emp_name,
       salary,
       ROW_NUMBER() OVER(ORDER BY salary DESC) AS row_no
FROM Employee;
```

Output

Name	Salary	Row Number
Neha	95000	1
Kavita	88000	2
Sneha	85000	3
Karan	80000	4
Priya	75000	5

---

4. Window Function - DENSE_RANK()

Query

```
SELECT emp_id,
       emp_name,
       salary,
       DENSE_RANK() OVER(ORDER BY salary DESC) AS dense_rank
FROM Employee;
```

Output

Name	Salary	Dense Rank
Neha	95000	1
Kavita	88000	2
Sneha	85000	3
Karan	80000	4
Priya	75000	5

---

5. CASE WHEN

Query

```
SELECT emp_name,
       salary,
       CASE
```

```
           WHEN salary >= 80000 THEN 'High Salary'
           WHEN salary >= 60000 THEN 'Medium Salary'
           ELSE 'Low Salary'
       END AS Salary_Category
FROM Employee;
```

Output

Employee	Salary	Category
Neha	95000	High Salary
Kavita	88000	High Salary
Sneha	85000	High Salary
Karan	80000	High Salary
Priya	75000	Medium Salary
Rahul	65000	Medium Salary
Amit	50000	Low Salary
Vikas	45000	Low Salary

---

6. Create View

Query

```
CREATE VIEW EmployeeDetails AS
SELECT e.emp_id,
       e.emp_name,
       e.salary,
       d.dept_name
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id;
```

Output

```
View EmployeeDetails created successfully.
```

---

7. Display View Data

Query

```
SELECT * FROM EmployeeDetails;
```

Output

Emp ID	Name	Salary	Department
101	Amit	50000	HR
102	Priya	75000	IT

Emp ID	Name	Salary	Department
103	Rahul	65000	IT
104	Sneha	85000	Finance
105	Vikas	45000	HR
106	Neha	95000	Finance
107	Rohit	70000	Marketing
108	Pooja	60000	Marketing
109	Karan	80000	IT
110	Anjali	55000	HR

---

8. Create Audit Table

Query

```
CREATE TABLE Employee_Audit(
audit_id INT AUTO_INCREMENT PRIMARY KEY,
emp_id INT,
action_type VARCHAR(50),
action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Output

```
Table Employee_Audit created successfully.
```

---

9. Create Trigger

Query

```
CREATE TRIGGER trg_employee_insert
AFTER INSERT
ON Employee
FOR EACH ROW
INSERT INTO Employee_Audit(emp_id,action_type)
VALUES(NEW.emp_id,'INSERT');
```

Output

```
Trigger created successfully.
```

---

10. Test Trigger

Query

```
INSERT INTO Employee
VALUES(118,'Raj',70000,'2025-06-20',1);
```

Output

```
1 row inserted.
```

---

11. Verify Trigger Output

Query

```
SELECT * FROM Employee_Audit;
```

Output

Audit ID	Emp ID	Action Type	Action Time
1	118	INSERT	2025-06-20 10:30:00

SQL Interview Questions and Answers

---

1. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

ROW_NUMBER()

Assigns a unique number to each row.

Example

Salary	ROW_NUMBER
95000	1
95000	2
85000	3

Even if salaries are the same, row numbers are different.

RANK()

Assigns the same rank to duplicate values but skips the next rank.

Example

Salary	Rank
95000	1
95000	1
85000	3

Rank 2 is skipped.

DENSE_RANK()

Assigns the same rank to duplicate values and does not skip ranks.

Example

Salary	Dense Rank
95000	1
95000	1
85000	2

---

2. What is a Window Function?

Answer

A Window Function performs calculations across a set of rows related to the current row without grouping the result.

Examples

- RANK()
- DENSE_RANK()
- ROW_NUMBER()
- LEAD()
- LAG()
- SUM() OVER()
- AVG() OVER()

Syntax

```
Function_Name() OVER(
    PARTITION BY column_name
    ORDER BY column_name
)
```

Uses

- Ranking
- Running totals
- Moving averages
- Data analysis

---

3. Why do we use CASE WHEN?

Answer

CASE WHEN is used to implement conditional logic in SQL.

It works like an IF-ELSE statement in programming languages.

Example

```
CASE
    WHEN salary > 80000 THEN 'High'
    WHEN salary > 60000 THEN 'Medium'
    ELSE 'Low'
END
```

Uses

- Data categorization
- Conditional calculations
- Report generation

---

4. What is a View and why is it used?

Answer

A View is a virtual table created from one or more tables using a query.

It does not store data physically.

Example

```
CREATE VIEW EmployeeDetails AS
SELECT emp_name,salary
FROM Employee;
```

Advantages

- Security
- Simplifies complex queries

- Reusability
- Data abstraction

---

5. Can data be updated through a View?

Answer

Yes, but with some restrictions.

Updatable View

```
CREATE VIEW EmpView AS
SELECT emp_id, emp_name
FROM Employee;
UPDATE EmpView
SET emp_name='Raj'
WHERE emp_id=101;
```

Non-Updatable View

Views containing:

- GROUP BY
- Aggregate Functions
- DISTINCT
- UNION

cannot usually be updated.

---

6. What is a Trigger?

Answer

A Trigger is a database object that executes automatically when an event occurs on a table.

Events

- INSERT
- UPDATE
- DELETE

Example

```
CREATE TRIGGER trg_insert
AFTER INSERT
ON Employee
```

```
FOR EACH ROW
...
```

Uses

- Audit logging
- Validation
- Security
- Automatic updates

---

7. What are BEFORE and AFTER Triggers?

BEFORE Trigger

Runs before the event occurs.

Example

```
BEFORE INSERT
```

Uses

- Validation
- Data modification

AFTER Trigger

Runs after the event occurs.

Example

```
AFTER INSERT
```

Uses

- Audit logs
- Notifications
- History tracking

---

8. What is the purpose of an Audit Table?

Answer

An Audit Table stores records of database activities.

Example

Audit ID	Employee ID	Action
1	101	INSERT
2	102	UPDATE

Benefits

- Tracks changes
- Maintains history
- Supports compliance requirements
- Helps debugging

---

9. What is the difference between a Stored Procedure and a Trigger?

Stored Procedure	Trigger
Executed manually	Executed automatically
Called using EXEC	Called by database events
Can accept parameters	Cannot accept parameters directly
Used for business logic	Used for automation

Stored Procedure Example

```
CREATE PROCEDURE GetEmployees()
BEGIN
    SELECT * FROM Employee;
END;
```

Trigger Example

```
CREATE TRIGGER trg_insert
AFTER INSERT
ON Employee
FOR EACH ROW
...
```

---

10. Where are Window Functions used in Real Projects?

Banking Systems

Finding top customers by transaction amount.

E-Commerce

Ranking products by sales.

HR Systems

Finding highest-paid employees in each department.

Analytics

Calculating running totals and trends.

Example

```
SELECT emp_name,
       salary,
       RANK() OVER(
           PARTITION BY dept_id
           ORDER BY salary DESC
       ) AS rank_no
FROM Employee;
```

Frequently Asked Interview Follow-up Questions

What is PARTITION BY?

It divides rows into groups before applying a window function.

```
RANK() OVER(
PARTITION BY dept_id
ORDER BY salary DESC
)
```

Difference Between WHERE and HAVING?

WHERE	HAVING
Filters rows	Filters groups
Used before GROUP BY	Used after GROUP BY

Difference Between DELETE, TRUNCATE, and DROP?

DELETE	TRUNCATE	DROP
Deletes rows	Removes all rows	Removes table

DELETE	TRUNCATE	DROP
Can use WHERE	Cannot use WHERE	Removes structure
Can rollback	Limited rollback	Cannot recover easily

What is a Foreign Key?

A Foreign Key creates a relationship between two tables.

Example:

```
FOREIGN KEY(dept_id)
REFERENCES Department(dept_id)
```

It ensures referential integrity and prevents invalid department IDs in the Employee table.